

PONTIFICIA UNIVERSIDAD CATÓLICA DEL PERÚ
FACULTAD DE CIENCIAS E INGENIERÍA

PROGRAMACIÓN 2
7ma práctica (tipo b)
Primer Semestre 2026

Indicaciones Generales:

- Duración: 110 minutos.

NO SE PERMITE EL USO DE APUNTES DE CLASE, FOTOCOPIAS NI MATERIAL IMPRESO

- No se pueden emplear **variables globales, NI OBJETOS**.
- **NO PUEDE UTILIZAR LA CLASE string**. Tampoco se podrán emplear las funciones malloc, realloc, memset, strtok o strdup, igualmente no se puede emplear cualquier función contenida en las bibliotecas stdio.h, cstdio o similares y que puedan estar también definidas en otras bibliotecas. **NO PODRÁ EMPLEAR PLANTILLAS EN ESTE LABORATORIO**
- Deberá modular correctamente el proyecto en archivos independientes. LAS SOLUCIONES DEBERÁN DESARROLLARSE BAJO UN ESTRICTO DISEÑO DESCENDENTE. Cada función **NO debe sobrepasar las 20 líneas de código aproximadamente**. El archivo main.cpp solo podrá contener la función main de cada proyecto y el código contenido en él solo podrá estar conformado por tareas implementadas como funciones. En el archivo main.cpp deberá colocar un comentario en el que coloque claramente su nombre y código, **de no hacerlo se le descontará 0.5 puntos en la nota final**.
- El código comentado **NO SE CALIFICARÁ**. De igual manera **NO SE CALIFICARÁ** el código de una función si esta función no es llamada en ninguna parte del proyecto o su llamado está comentado.
- Los programas que presenten errores de sintaxis o de concepto se calificarán en base al 40% de puntaje de la pregunta. Los que no muestren resultados o que estos no sean coherentes en base al 60%.
- Se tomará en cuenta en la calificación el uso de comentarios relevantes.
- **TAMPOCO SE PODRÁ EMPLEAR LA CLÁUSULA protected NI LA CLÁUSULA friend, DE HACERLO NO SE LE CALIFICARÁN LAS CLASES INVOLUCRADAS.**

SE LES RECUERDA QUE, DE ACUERDO AL REGLAMENTO DISCIPLINARIO DE NUESTRA INSTITUCIÓN, CONSTITUYE UNA FALTA GRAVE COPIAR DEL TRABAJO REALIZADO POR OTRA PERSONA O COMETER PLAGIO.

NO SE HARÁN EXCEPCIONES ANTE CUALQUIER TRASGRESIÓN DE LAS INDICACIONES DADAS EN LA PRUEBA

Puntaje total: 20 puntos.

INDICACIONES INICIALES

Cree un proyecto de C++ en CLion siguiendo estrictamente las indicaciones que a continuación se detallan:

- La unidad de trabajo será **t:** (Si lo coloca en otra unidad, no se calificará su laboratorio y se le asignará como nota cero)
- Cree allí una carpeta con el nombre **"CO_PA_PN_Lab07_2026_1"** donde **CO** indica: Código del alumno, **PA** indica: Primer Apellido del alumno y **PN** primer nombre (de no colocar este requerimiento se le descontará 3 puntos de la nota final). **Allí colocará el proyecto solicitado en la prueba.**

Cuestionario:

La finalidad principal de este laboratorio es la de reforzar los conceptos contenidos en el capítulo 3 del curso: "Programación Orientada a Objetos". En este laboratorio se trabajará con Polimorfismo de Objetos.

Deberá elaborar un proyecto denominado **"EmergenciasPROG2"** y en él desarrollará el programa que dé solución al problema planteado. **DE NO COLOCAR ESTE REQUERIMIENTO SE LE DESCONTARÁ 3 PUNTOS DE LA NOTA FINAL. NO SE HARÁN EXCEPCIONES**

NO PODRÁ EMPLEAR ARREGLOS DE MÁS DE UNA DIMENSIÓN

NO PUEDE MANIPULAR UN PUNTERO CON MÁS DE UN ÍNDICE

LOS ARCHIVOS SOLO SE PUEDEN LEER UNA VEZ

Se le ha contratado para desarrollar un sistema de gestión de atenciones médicas para la Red Nacional de Emergencias. El objetivo es registrar, clasificar y reportar todas las atenciones realizadas en un centro médico durante una jornada completa (24 horas), aplicando los principios de herencia, encapsulamiento y polimorfismo en programación orientada a objetos.

Dentro del sistema existirán tres tipos principales de atenciones:

- **Emergencias**, que corresponden a situaciones críticas que ponen en riesgo inmediato la vida del paciente y requieren intervención médica inmediata. Algunos ejemplos son paros cardiorrespiratorios, traumatismos severos, hemorragias masivas o shock anafiláctico.
- **Urgencias**, que corresponden a situaciones que requieren atención médica rápida, aunque no comprometen de forma inmediata la vida del paciente. Algunos ejemplos son fracturas, heridas profundas, crisis asmáticas controladas o lesiones deportivas.
- **No Urgencias**, que corresponden a consultas médicas programadas o afecciones que no representan un riesgo inmediato para la salud del paciente. Algunos ejemplos son controles médicos, consultas dermatológicas, chequeos preventivos y seguimientos de tratamiento.

El sistema deberá permitir almacenar información específica para cada tipo de atención médica y generar reportes diarios que permitan analizar la distribución de pacientes atendidos según su nivel de prioridad.

La información no se encuentra almacenada en un único archivo. Debido al gran volumen de registros generados diariamente, las atenciones se encuentran organizadas jerárquicamente según la fecha y la hora en la que fueron registradas.

La estructura de almacenamiento es la siguiente:

```
Datos/  
└── Año/  
    └── Mes/  
        └── Día/  
            ├── 00.csv  
            ├── 01.csv  
            ├── 02.csv  
            ├── ...  
            └── 23.csv
```

Cada archivo representa todas las atenciones registradas durante una hora específica del día.

Por ejemplo:

Datos/2026/06/15/08.csv, contiene todas las atenciones registradas entre las 08:00 y las 08:59 horas del día 15 de junio de 2026.

Datos/2026/01/01/01.csv, contiene todas las atenciones registradas entre las 01:00 y las 01:59 horas del 01 de enero de 2026.

El sistema deberá ser capaz de procesar automáticamente los veinticuatro archivos correspondientes a una fecha determinada, cargando toda la información necesaria para generar los reportes solicitados.

Cada archivo CSV contiene una cantidad variable de registros y presenta la siguiente estructura:

Datos/2026/06/12/10.csv				
1	1001	Juan Perez	Paro Cardiaco	1,3
2	1002	Ana Ruiz	Dificultad Respiratoria	Torax,1
3	1003	Carlos Diaz	Dermatitis	Dermatologia,0
...				

Los tipos de atención se representan mediante los siguientes códigos: 1 : Emergencia, 2 : Urgencia y 3 : NoUrgenciaLa estructura es la siguiente:

tipo_atencion, codigo_atencion, paciente, descripcion

Luego puede venir, dependiendo del tipo de atención:

Emergencia: requiere_rcp, minutos_sin_pulso

Urgencia: zona_lesion, necesita_rx

NoUrgencia: especialidad, es_consulta_program

PARTE 1 (6 puntos)

Construya una clase abstracta denominada **Atencion**, que represente cualquier atención médica registrada dentro del sistema.

Para ello añada los siguientes atributos privados:

1. **codigo**, entero (int) que almacena el código único de la atención.
2. **paciente**, cadena dinámica (char*) que almacena el nombre completo del paciente.
3. **descripcion**, cadena dinámica (char*) que almacena una breve descripción del motivo de atención.
4. **hora**, entero (int) que indica la hora en la que fue registrada la atención.

Implemente **OBLIGATORIAMENTE**, los constructores por defecto, constructores copia, destructores, getters, setters y sobrecargas necesarias.

Además implemente los siguientes métodos:

- **leer**: permite leer la información común de una atención desde un archivo CSV y almacenarla en el objeto. Esta función también debe guardar la hora de la atención, si se está procesando el archivo 14.csv, quiere decir que el valor del atributo hora es 14.
- **imprimir**: muestra la información de la atención en una sola línea correctamente formateada.
- **obtener_prioridad**: método virtual puro que permitirá determinar la prioridad asociada a cada tipo de atención. Emergencia prioridad 1, Urgencia prioridad 2 y NoUrgencia prioridad 3.

Implemente las siguientes clases derivadas:

Emergencia: requiere_rcp (bool) y minutos_sin_pulso (int)

Urgencia: zona_lesion (char*) y necesita_rx (bool)

NoUrgencia: especialidad (char*) y es_consulta_programada (bool)

Cada clase derivada deberá:

- Implementar sus propios constructores, destructores, getters y setters.
- Sobrescribir los métodos leer e imprimir para incluir la información específica de cada tipo de atención.
- Implementar adecuadamente el método obtenerPrioridad().

PARTE 2 (4 puntos)

Construya una clase denominada **GestorAtenciones**, la cual será responsable de administrar todas las atenciones registradas durante una jornada.

La clase deberá contener los siguientes atributos:

1. **atenciones**, arreglo dinámico de punteros a objetos de tipo Atencion. Si desea tambien puede definirlo como un arreglo estático de punteros a objetos Atención, bajo su propio riesgo.
2. **cantidad_atenciones**, entero que almacena la cantidad actual de atenciones registradas.
3. **capacidad**, entero que almacena la capacidad actual del arreglo dinámico.

Implemente **OBLIGATORIAMENTE**, los constructores por defecto, constructores copia, destructores, getters, setters y sobrecargas necesarias.

Además implemente los siguientes métodos:

reservar_memoria(): Incrementa la capacidad del arreglo dinámico cuando sea necesario, conservando la información previamente almacenada.

reservar_memoria_atencion(char tipo): Reserva memoria para el tipo de atención correspondiente y retorna un puntero a la clase base Atencion. Este método deberá permitir aprovechar el polimorfismo durante la carga de datos.

agregar_atencion(Atencion *) : Inserta una nueva atención dentro de la colección administrada por el gestor.

cargar_atenciones(const char *nombre_archivo, int hora): Lee todas las atenciones contenidas en el archivo especificado por nombre_archivo y las almacena dentro del sistema.

PARTE 3 (10 puntos)

Implemente los siguientes métodos dentro de la clase **GestorAtenciones**:

procesar_dia(const char* fecha)

Este método deberá procesar todas las atenciones registradas durante la fecha indicada, el formato que recibe es "2026-06-12"

Para ello deberá recorrer automáticamente los veinticuatro archivos correspondientes al día solicitado:

Datos/2026/06/12/00.csv

Datos/2026/06/12/01.csv

..

Datos/2026/06/12/23.csv

generar_reporte_diario(const char *nombre_reporte)

Genera un reporte consolidado con todas las atenciones procesadas durante el día.

El reporte deberá agrupar las atenciones según su nivel de prioridad médica, mostrando tres secciones claramente diferenciadas.

EMERGENCIAS:> Prioridad 1

URGENCIAS :> Prioridad 2

NOURGENCIAS :> Prioridad 3

La información deberá mostrarse utilizando polimorfismo, es decir, invocando el método imprimir() a través de punteros de tipo Atencion.

```

=====
HORA: 00:00
=====
EMERGENCIAS:
=====
CODIGO  NOMBRES      DESCRIPCION      RCP?  NO_PULSO      OBSERVACION
1001    Juan Perez      Paro Cardiaco    SI     3              Nivel: CRITICO ( 0 min)
1002    Ana Ruiz        Dificultad Respiratoria  SI     1              Nivel: CRITICO ( 0 min)
1004    Maria Lopez     Migrana          NO     0              Nivel: CRITICO ( 5 min)
=====
URGENCIAS:
=====
CODIGO  NOMBRES      DESCRIPCION      ZONA_LESION  RX?      OBSERVACION
1003    Carlos Diaz    Fractura Brazo   Derecho      SI        Tiempo max: 30 min
=====
NO URGENCIAS:
=====
CODIGO  NOMBRES      DESCRIPCION      ESPECIALIDAD  CONSULTA_PROGRAMADA?
1005    Luis Torres    Dermatitis       Dermatologia  NO
=====
HORA: 01:00
=====
...
Modelo reporte Atenciones por Hora

```

Al finalizar la práctica, comprima la carpeta dada en las indicaciones iniciales empleando el programa Zip que viene por defecto en el Windows, no se aceptarán los trabajos compactados con otros programas como RAR, WinRAR, 7zip o similares.

Profesores del curso: Miguel Guanira Ana Roncal
 Rony Cueva Eric Huiza
 Erasmo Gómez

San Miguel, 12 de Junio del 2026.